

KONERU LAKSHMAIAH EDUCATION FOUNDATION
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

PROJECT REPORT
AI PERSONAL HEALTH SYSTEM

Submitted in partial fulfillment of the requirements for the award of the
degree of
BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING

Submitted By
G. Tej Sahith – 2520030606
A. Sreepad – 2520030152
Kambhammettu Srithika – 2520030361

Under the Guidance of
Dr. Y. Subbarayudu
Assistant Professor
Department of Computer Science and Engineering

GROUP NUMBER: 1
Academic Year: 2026–2027

DECLARATION

We hereby declare that the project report entitled “AI Personal Health System” is a record of the work carried out by us as part of our academic project requirements in the Department of Computer Science and Engineering, K L (Deemed to be) University.

The project has been developed under the guidance of Dr. Y. Subbarayudu, Assistant Professor, Department of Computer Science and Engineering. The work presented in this report represents the project concept, architecture, implementation plan, and technical work undertaken by the project team.

We further declare that the report has been prepared for academic evaluation and that the technical descriptions have been organized according to the project review material and the prescribed documentation format.

G. Tej Sahith – 2520030606

A. Sreepad – 2520030152

Kambhammettu Srithika – 2520030361

Place: Hyderabad

Date: _____

CERTIFICATE

This is to certify that the project report entitled “AI Personal Health System” is a bonafide work carried out by G. Tej Sahith (2520030606), A. Sreepad (2520030152), and Kambhammettu Srithika (2520030361) in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering at K L (Deemed to be) University.

The project has been carried out under the guidance and supervision of Dr. Y. Subbarayudu, Assistant Professor, Department of Computer Science and Engineering.

Guide Signature: _____

Course Coordinator: _____

Head of Department: _____

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to everyone who supported us during the development of our project, “AI Personal Health System.”

We express our deepest gratitude to our project guide, Dr. Y. Subbarayudu, Assistant Professor, Department of Computer Science and Engineering, for his valuable guidance, continuous encouragement, technical suggestions, and support throughout the development of this project.

We are grateful to the Department of Computer Science and Engineering, K L (Deemed to be) University, for providing the academic environment and facilities required for carrying out the project.

We also thank all faculty members and technical staff who supported us during analysis, design, development, testing, presentation, and documentation.

Finally, we thank our parents, friends, classmates, and team members for their continuous motivation and cooperation.

ABSTRACT

The AI Personal Health System is a database-driven backend application designed to help users securely store, manage, and access personal health information in one centralized system. The system allows users to create accounts, maintain personal profiles, store health records, record symptoms and vital measurements, manage medication information, and view previous health history.

A relational database organizes the information using tables, relationships, constraints, and queries. The backend provides RESTful APIs for authentication, health-record management, retrieval, and updating information. Validation mechanisms and database transactions support consistency and integrity.

The project also includes an AI-based health insight concept that can analyze stored health information and provide general personalized health insights and recommendations for awareness. The system is intended for personal health management and awareness and is not a replacement for professional medical diagnosis.

The technology stack consists of Java with Spring Boot, RESTful Web Services, MySQL, JPA/JDBC, Maven, development environments such as VS Code or IntelliJ IDEA, and Postman for API testing.

TABLE OF CONTENTS

- 1. 1 Introduction
- 2. 2 System Requirements
- 3. 3 Technology Stack
- 4. 4 System Architecture
- 5. 5 Design
- 6. 6 Implementation
- 7. 7 Features
- 8. 8 Testing
- 9. 9 Deployment
- 10. 10 Challenges & Limitations
- 11. 11 Future Enhancements
- 12. 12 Conclusion
- 13. 13 References
- 14. 14 Appendices

Note: After opening this document in Microsoft Word, use References → Table of Contents → Update Table to generate exact page numbers if you modify the document.

CHAPTER 1 – INTRODUCTION

1.1 Background of the Project

Personal health information is generated throughout a person's life. It may include symptoms, vital measurements, medication information, appointment details, health records, and historical information. When such information is maintained manually or across separate locations, users may experience difficulty in organizing, retrieving, and reviewing their own records.

The AI Personal Health System is proposed as a centralized backend platform for managing this information. The project review describes a database-driven system that allows users to create accounts, maintain profiles, store health records, record symptoms and measurements, manage medication information, manage appointments, and view health history.

The system combines application development with relational database management. Java and Spring Boot form the backend foundation, while MySQL provides persistent relational storage. RESTful APIs provide a standardized communication mechanism between clients and backend services.

An AI-based module is included to support general personalized health insights. The purpose of this component is health awareness and self-management rather than professional diagnosis.

CHAPTER 1 – INTRODUCTION

1.2 Problem Statement

Health information is often fragmented across notebooks, paper records, files, reports, and separate applications. This fragmentation can make it difficult to maintain an accurate and continuously accessible record.

The absence of centralized relationships can also make it difficult to connect a symptom with a measurement, medication, appointment, or historical record. A system that stores information in a structured relational database can provide a consistent foundation for retrieving and updating records.

The project therefore addresses the need for a reliable backend system that stores personal health information and provides APIs for managing it. The system also addresses the need for general health-awareness insights by introducing an AI-based analysis component.

Problem Area	Effect	System Response
Fragmented records	Difficult retrieval	Centralized database
Manual updates	Inconsistent information	Structured CRUD operations
Unrelated records	Difficult history review	Relational relationships
Limited analysis	Low awareness of trends	AI-based general insights
Multiple data sources	Hard to maintain consistency	Validation and transactions

CHAPTER 1 – INTRODUCTION

1.3 Scope of the Project

The scope of the project is focused on personal health-data management through a centralized backend. It includes account and profile management, health records, symptoms, vital measurements, medications, appointments, health history, REST APIs, relational database management, validation, transactions, and AI-based health insights.

- Centralized personal health-data storage.
- Structured management of health records.
- Symptom and vital-measurement recording.
- Medication and appointment management.
- Historical health-information retrieval.
- RESTful API services.
- MySQL relational database.
- Validation and transaction support.
- AI-based general health-awareness insights.

The scope does not include replacing clinical professionals or providing a certified medical diagnosis. Future interfaces and external integrations are considered extensions of the current backend-oriented architecture.

CHAPTER 1 – INTRODUCTION

1.4 Objectives

1. Centralize personal health information in one system.
2. Allow users to manage profiles and health records.
3. Store symptoms, vital measurements, medications, appointments, and health history.
4. Design a structured relational database with appropriate relationships.
5. Provide RESTful APIs for health-data operations.
6. Maintain data consistency using validation and transactions.
7. Provide general AI-based health insights for awareness.
8. Create an extensible backend foundation for future client applications.
9. Demonstrate practical use of Java, Spring Boot, MySQL, REST APIs, and database-access technologies.

These objectives correspond to the project review's stated contribution and objective areas.

CHAPTER 1 – INTRODUCTION

1.5 Importance of the Application

The importance of the system can be considered from both the user and software-engineering perspectives. From the user perspective, centralized storage can make personal health information easier to organize and review.

From the engineering perspective, the project demonstrates the integration of multiple backend technologies. It shows how a Java application can communicate with a relational database, expose REST APIs, validate information, manage transactions, and support an AI-based processing component.

The project also establishes a foundation for future features such as dashboards, reminders, wearable integration, mobile applications, and healthcare-service integration. These extensions are identified as future scope rather than being assumed to be completed in the current version.

CHAPTER 1 – INTRODUCTION

1.6 Target Users / Audience

Target Group	Purpose
Individual users	Maintain and review personal health information.
Students / developers	Study backend, database, REST API, and AI integration concepts.
Future mobile/web users	Access health records through client applications.
Health-awareness users	Receive general informational insights from stored data.

The primary audience is individuals interested in organized personal health management. The application should clearly communicate that AI-generated information is general and should not be interpreted as professional medical diagnosis.

CHAPTER 1 – INTRODUCTION

1.7 Existing System and Proposed System

Aspect	Existing / Fragmented Approach	Proposed System
Storage	Manual or separate records	Centralized relational database
Retrieval	Manual searching	API-based retrieval
Relationships	Often disconnected	Structured relational relationships
Updates	May be inconsistent	Controlled backend operations
Analysis	Limited manual interpretation	AI-based general insights
Extensibility	Depends on separate tools	REST-based backend foundation

The project review identifies manual or fragmented record keeping as a major limitation and proposes a centralized backend as the contribution. The proposed system is therefore designed around structured storage, APIs, validation, transactions, and AI-based general insights.

CHAPTER 2 – SYSTEM REQUIREMENTS

2.1 Hardware Requirements

The system is primarily a backend and database application. Development requires a computer capable of running Java, Spring Boot, MySQL, an IDE, and Postman.

Component	Recommended Requirement
Processor	Intel Core i5 / equivalent multi-core processor or better
RAM	8 GB minimum; 16 GB recommended
Storage	At least 20–50 GB free space
Display	1366 × 768 or higher
Network	Stable internet connection
Input	Keyboard and mouse

The exact hardware requirement may vary depending on the operating system, IDE, database size, and number of development tools running simultaneously.

CHAPTER 2 – SYSTEM REQUIREMENTS

2.2 Software Requirements

Software	Role
Java / JDK	Backend programming and execution
Spring Boot	Backend framework
MySQL	Relational database
JPA / JDBC	Database access
Maven	Build and dependency management
VS Code / IntelliJ IDEA	Development
Postman	API testing
Web Browser	Future client/API interaction

These technologies are based on the software requirements identified in the project review.

CHAPTER 2 – SYSTEM REQUIREMENTS

2.3 Functional Requirements

10. The system shall allow a user to create an account.
11. The system shall support authentication-related operations.
12. The system shall maintain personal profile information.
13. The system shall allow health records to be created and retrieved.
14. The system shall allow symptoms to be recorded.
15. The system shall allow vital measurements to be recorded.
16. The system shall support medication information.
17. The system shall support appointment information.
18. The system shall provide access to health history.
19. The system shall expose RESTful APIs.
20. The system shall validate appropriate input.
21. The system shall maintain database consistency.
22. The system shall support general AI-based health insights.

CHAPTER 2 – SYSTEM REQUIREMENTS

2.4 Non-Functional Requirements

Requirement	Description
Security	Protect account and health information from unauthorized access.
Reliability	Provide consistent behavior for valid operations.
Maintainability	Use modular backend organization.
Scalability	Allow future clients and modules to be added.
Usability	Provide clear API behavior and structured responses.
Data Integrity	Maintain valid relationships and consistent records.
Performance	Process normal API requests efficiently.
Extensibility	Support future mobile, wearable, and AI enhancements.

CHAPTER 2 – SYSTEM REQUIREMENTS

2.5 Constraints and Assumptions

The project assumes that users provide their own health information accurately. The system cannot independently verify every health value entered by a user.

The system also assumes that the MySQL database is available when persistent operations are performed. Backend functionality depends on correct database configuration and valid application settings.

The AI component is assumed to provide general informational output. The project does not claim that AI-generated output is equivalent to clinical diagnosis.

The exact hardware, deployment environment, server configuration, and final API endpoints should be documented according to the team's final implementation.

CHAPTER 3 – TECHNOLOGY STACK

3.1 Java

Java is the primary programming language used for the backend. Its object-oriented model supports the representation of project concepts as classes and objects. Java also provides exception handling, strong type checking, reusable libraries, and an established ecosystem for server-side development.

The use of Java is appropriate for a modular backend because domain entities, services, controllers, and repositories can be separated into distinct responsibilities.

CHAPTER 3 – TECHNOLOGY STACK

3.2 Spring Boot

Spring Boot provides the application framework for the backend. It simplifies configuration and supports the development of RESTful web services.

A typical Spring Boot architecture separates controllers, services, and data-access components. Controllers handle HTTP requests, services contain application logic, and repositories or JDBC components communicate with the database.

This layered organization improves maintainability and allows future client applications to communicate with the same backend.

CHAPTER 3 – TECHNOLOGY STACK

3.3 RESTful Web Services

RESTful Web Services provide the communication interface between clients and the backend. REST uses standard HTTP methods to perform operations on resources.

Method	Purpose	Typical Health Use
GET	Retrieve	Retrieve health records
POST	Create	Create symptom or appointment
PUT	Update	Update profile or health record
DELETE	Delete	Remove a permitted record

REST architecture allows the backend to remain independent from the user interface. A future web application and mobile application can both consume the same REST endpoints.

CHAPTER 3 – TECHNOLOGY STACK

3.4 MySQL

MySQL is used as the relational database. A relational model is appropriate because the system contains structured entities and relationships.

Primary keys identify records, while foreign keys can associate health records with users. Constraints can reduce invalid or inconsistent database states.

The database provides persistent storage so that information remains available beyond a single application session.

CHAPTER 3 – TECHNOLOGY STACK

3.5 JPA / JDBC

JPA/JDBC provides database-access capabilities between the Java application and MySQL. JPA can map Java entities to database tables and simplify common persistence operations. JDBC provides a lower-level mechanism for connecting to the database and executing SQL.

The project review explicitly lists JPA/JDBC as the database-access technology. The final implementation may use one or both depending on the source-code design.

CHAPTER 3 – TECHNOLOGY STACK

3.6 Maven, IDE and Postman

Maven manages project dependencies and the build lifecycle. It helps compile source code, execute tests, package the application, and maintain a reproducible project configuration.

VS Code and IntelliJ IDEA are identified as development environments. Postman is used to test REST APIs by sending requests and inspecting responses.

Tool	Main Use
Maven	Build and dependency management
VS Code	Source-code development
IntelliJ IDEA	Java development and debugging
Postman	API testing
MySQL tools	Database inspection

CHAPTER 4 – SYSTEM ARCHITECTURE

4.1 High-Level Architecture

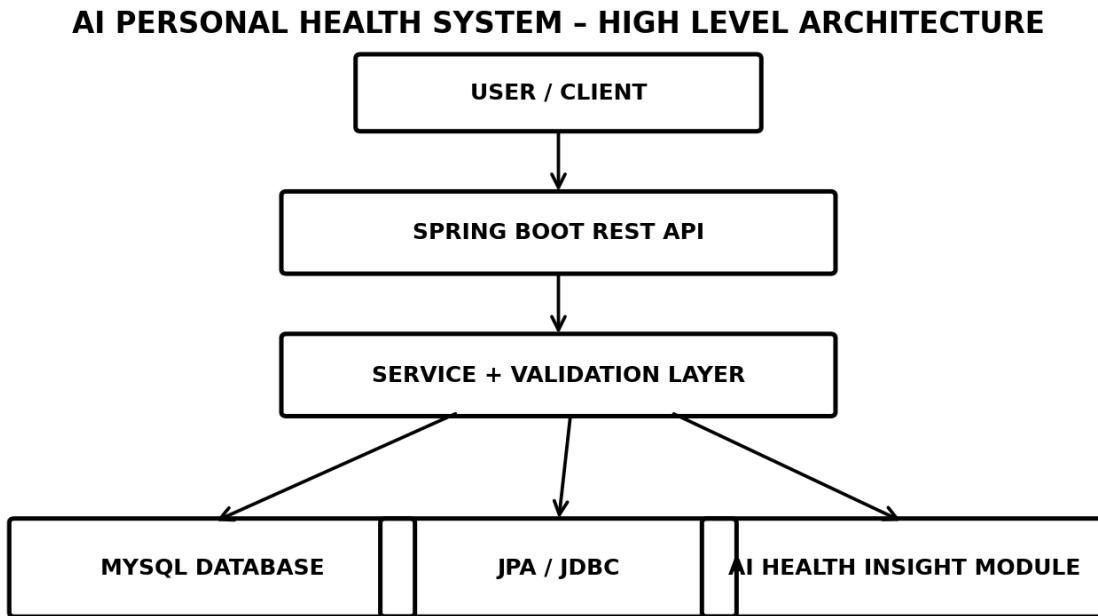


Figure 4.1: High-level architecture

The architecture follows a layered model consisting of a client, REST API layer, business-logic and validation layer, database-access layer, MySQL database, and AI health-insight component.

CHAPTER 4 – SYSTEM ARCHITECTURE

4.2 Client and API Layers

The client layer represents a future or existing user interface that communicates with the backend. The project review primarily specifies the backend system, so this documentation treats the client as a consuming layer rather than claiming a completed frontend implementation.

The REST API layer receives requests, identifies the requested resource, validates the request, and passes processing to the service layer.

A client should not need to know how data is stored internally. It communicates through the documented API contract.

CHAPTER 4 – SYSTEM ARCHITECTURE

4.3 Service and Validation Layer

The service layer contains application-level rules. It can verify that the authenticated user is associated with a requested record, coordinate multiple database operations, and prepare information for persistence.

Validation is important because health data should not be stored without basic checks. Validation can include required fields, accepted formats, numerical boundaries where defined, and relationship checks.

Transactions can be applied to operations that involve multiple dependent changes.

CHAPTER 4 – SYSTEM ARCHITECTURE

4.4 Database Layer

The database layer is responsible for persistent storage. JPA/JDBC communicates with MySQL and maps application operations to database operations.

The database should maintain clear relationships between a user and that user's health information. This allows the system to retrieve personal records in an organized manner.

Indexes can be considered for frequently queried columns in a future optimization phase, but the actual indexes should be documented only if they are present in the final implementation.

CHAPTER 4 – SYSTEM ARCHITECTURE

4.5 AI Insight Layer

The AI health-insight layer receives relevant health information and produces general personalized insights. The module is intended for awareness and self-management.

The AI layer should be treated as an information-processing component rather than a medical authority. The system should clearly distinguish generated insights from professional diagnosis.

Future versions may extend the AI layer with more advanced analytics, but such functionality is identified as future scope in the project review.

CHAPTER 4 – SYSTEM ARCHITECTURE

4.6 Data Flow

23. User sends a request.
24. REST controller receives the request.
25. Input is validated.
26. Service layer applies application rules.
27. JPA/JDBC performs database operations where required.
28. MySQL stores or retrieves information.
29. AI analysis is invoked for insight requests.
30. Backend prepares a structured response.
31. Client receives the response.

This flow illustrates the logical interaction between the major system layers.

CHAPTER 5 – DESIGN

5.1 Design Principles

- Modularity – separate major functions into manageable components.
- Separation of concerns – keep API, business logic, persistence, and AI processing responsibilities distinct.
- Data integrity – use relationships and constraints.
- Validation – verify incoming information before persistence.
- Security – restrict access to user-specific records.
- Maintainability – use a consistent backend structure.
- Extensibility – allow future interfaces and integrations.

These principles support a backend that can be extended without redesigning the entire application.

CHAPTER 5 – DESIGN

5.2 Logical Database Model

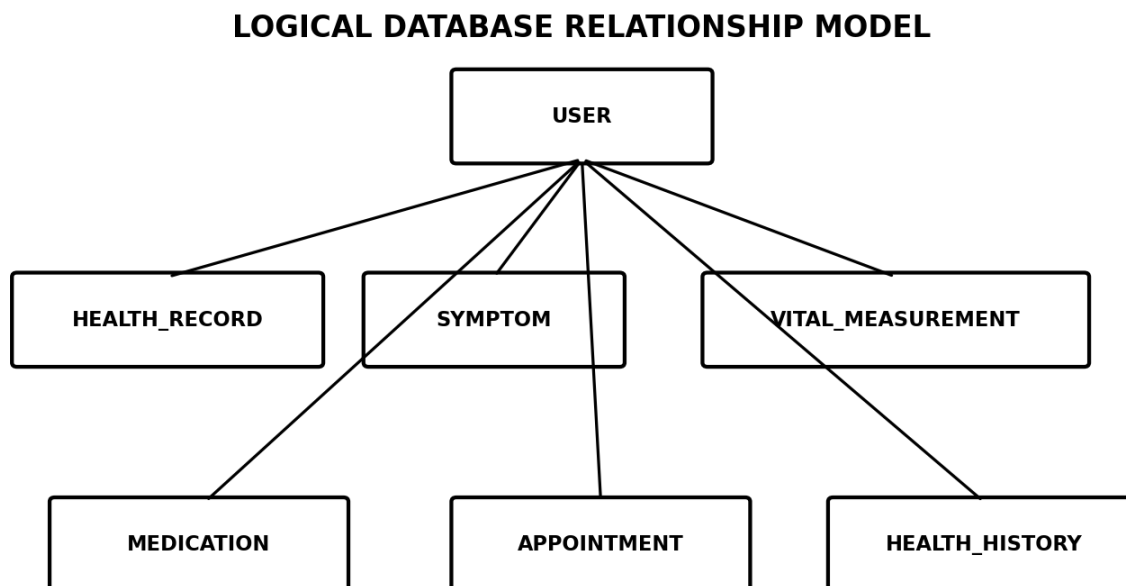


Figure 5.1: Logical relationship model

The diagram is a documentation-level logical model derived from the project modules. The exact field names and table definitions must match the final database implementation.

CHAPTER 5 – DESIGN

5.3 Entity Descriptions

Entity	Purpose
USER	Account and profile information
HEALTH_RECORD	General health records
SYMPTOM	Recorded symptoms
VITAL_MEASUREMENT	Recorded measurements
MEDICATION	Medication information
APPOINTMENT	Appointment information
HEALTH_HISTORY	Historical health information
AI INSIGHT	Generated general health-awareness output

The entities above represent the conceptual modules listed in the project review.

CHAPTER 5 – DESIGN

5.4 Relationships

The USER entity acts as the central identity for personal records. A user can have multiple health records, symptoms, measurements, medications, appointments, and historical records.

Relationship	Conceptual Cardinality
User → Health Records	One-to-Many
User → Symptoms	One-to-Many
User → Vital Measurements	One-to-Many
User → Medications	One-to-Many
User → Appointments	One-to-Many
User → Health History	One-to-Many

These cardinalities are logical documentation assumptions based on the modules; the final SQL schema should be used as the authoritative source.

CHAPTER 5 – DESIGN

5.5 API Design

API Group	Purpose
Authentication	Account and authentication operations
Profile	Personal profile operations
Health Records	Health record CRUD operations
Symptoms	Symptom operations
Vitals	Vital measurement operations
Medications	Medication operations
Appointments	Appointment operations
History	Historical information
AI Insights	General health insight requests

The exact endpoint paths should be copied from the final source code before submission.

CHAPTER 5 – DESIGN

5.6 Input and Output Design

REST API input should use structured request bodies for operations that create or update resources. Output should provide structured response information that can be consumed by a web or mobile client.

Error responses should clearly communicate validation or authorization failures without exposing sensitive implementation details.

Response Category	Meaning
Success	Requested operation completed
Validation Error	Input did not satisfy required conditions
Authentication Error	Credentials or authentication information invalid
Authorization Error	User is not permitted to access resource
Not Found	Requested resource does not exist
Server Error	Unexpected backend failure

CHAPTER 5 – DESIGN

5.7 Security and Privacy Design

Health information is sensitive, so the system should be designed with privacy and access control in mind. Authentication establishes the identity of the user, while authorization determines whether the user can access a specific resource.

- Do not expose database credentials in source code.
- Restrict user-specific health records to authorized users.
- Validate incoming request data.
- Use secure configuration for credentials.
- Do not present AI-generated insights as medical diagnosis.
- Protect sensitive information during storage and transmission.

CHAPTER 6 – IMPLEMENTATION

6.1 Module-Wise Implementation

The project consists of eight major modules identified in the project review. These modules divide the system into manageable functional responsibilities.

Module	Responsibility
Authentication	Account access
Profile	Personal information
Health Records	Structured health information
Symptoms & Vitals	Measurements and symptoms
Medication	Medication information
Appointments & History	Scheduling and historical records
Database	Persistent storage
REST APIs	Communication
AI	General health insights

CHAPTER 6 – IMPLEMENTATION

6.2 Authentication and Profile Management

The authentication module manages account-related operations. The profile component stores personal information associated with the authenticated account.

32. Receive authentication request.
33. Validate account information.
34. Authenticate the user.
35. Allow access to permitted resources.
36. Associate future records with the authenticated user.

CHAPTER 6 – IMPLEMENTATION

6.3 Health Record Management

Health records provide the main structured storage for personal health information. Records are associated with users through database relationships.

- Receive health-record request.
- Validate input.
- Associate record with the user.
- Persist record using JPA/JDBC.
- Return structured response.

CHAPTER 6 – IMPLEMENTATION

6.4 Symptoms Module

The symptoms module records information entered by users about symptoms. Structured symptom records can later contribute to historical review and AI analysis.

The module is designed to keep symptom information separate from other records while still maintaining a relationship with the corresponding user.

CHAPTER 6 – IMPLEMENTATION

6.5 Vital Measurements Module

The vital-measurement module stores measurement information in structured records. Historical measurements provide a foundation for future trend visualization.

Stage	Operation
1	Receive measurement
2	Validate value
3	Associate with user
4	Persist
5	Retrieve for history

CHAPTER 6 – IMPLEMENTATION

6.6 Medication Module

The medication module organizes medication-related information. Exact fields should follow the final schema.

A medication record may contain medication-related fields defined by the final database schema. The documentation should be updated with the exact fields after confirming the implemented SQL table.

CHAPTER 6 – IMPLEMENTATION

6.7 Appointment and History Module

Appointments provide structured appointment information, while health history allows users to review previous information.

The appointment and history modules extend the personal record beyond individual measurements. They support a chronological view of health-related information.

CHAPTER 6 – IMPLEMENTATION

6.8 REST API Implementation

Controllers receive HTTP requests and delegate processing to service components. REST methods should follow standard HTTP semantics.

Method	Operation
GET	Read
POST	Create
PUT	Update
DELETE	Delete where supported

CHAPTER 6 – IMPLEMENTATION

6.9 Validation and Transactions

Validation protects the database from invalid input. Transactions help keep related database operations consistent.

- Check required fields.
- Check valid formats.
- Verify user-resource relationships.
- Use transactions for related operations.
- Return meaningful error responses.

CHAPTER 6 – IMPLEMENTATION

6.10 AI Health Insight Implementation

The AI module processes relevant stored health information and returns general awareness-oriented insights.

37. Identify relevant user data.
38. Prepare information for analysis.
39. Process information using the implemented AI approach.
40. Generate general insight.
41. Return the result to the client.

CHAPTER 7 – FEATURES

7.1 Main Features

The main features include user authentication, profile management, health records, symptoms, vital measurements, medication management, appointments, health history, REST APIs, relational storage, validation, transactions, and AI-based general insights.

- Centralized information
- Structured records
- API access
- Data validation
- AI-based general insights

CHAPTER 7 – FEATURES

7.2 User Perspective

From a user perspective, the system provides a centralized place for personal health information. The user can maintain records and retrieve them through an application consuming the backend APIs.

- Centralized information
- Structured records
- API access
- Data validation
- AI-based general insights

CHAPTER 7 – FEATURES

7.3 Technical Perspective

Technically, the project integrates Java, Spring Boot, MySQL, JPA/JDBC, Maven, RESTful services, validation, transactions, and AI-based processing.

- Centralized information
- Structured records
- API access
- Data validation
- AI-based general insights

CHAPTER 7 – FEATURES

7.4 Feature Summary

The combination of these features provides a modular foundation for personal health-data management. The architecture is intentionally extensible so future web, mobile, wearable, and healthcare integrations can be added.

- Centralized information
- Structured records
- API access
- Data validation
- AI-based general insights

CHAPTER 8 – TESTING

8.1 Testing Strategy

Testing verifies that the application behaves correctly for valid and invalid inputs and that the API, service, database, and AI components interact correctly.

ID	Test	Expected Result	Actual Status
TC01	Valid request	Success response	Fill after execution
TC02	Invalid request	Validation error	Fill after execution
TC03	Unauthorized request	Access denied	Fill after execution
TC04	Database operation	Correct persistence	Fill after execution
TC05	AI request	Expected insight structure	Fill after execution

CHAPTER 8 – TESTING

8.2 Postman Testing

Postman is used to send requests to REST endpoints and inspect status codes, response bodies, headers, and error behavior.

ID	Test	Expected Result	Actual Status
TC01	Valid request	Success response	Fill after execution
TC02	Invalid request	Validation error	Fill after execution
TC03	Unauthorized request	Access denied	Fill after execution
TC04	Database operation	Correct persistence	Fill after execution
TC05	AI request	Expected insight structure	Fill after execution

CHAPTER 8 – TESTING

8.3 Authentication Testing

Authentication testing should include valid credentials, invalid credentials, missing information, and unauthorized access attempts.

ID	Test	Expected Result	Actual Status
TC01	Valid request	Success response	Fill after execution
TC02	Invalid request	Validation error	Fill after execution
TC03	Unauthorized request	Access denied	Fill after execution
TC04	Database operation	Correct persistence	Fill after execution
TC05	AI request	Expected insight structure	Fill after execution

CHAPTER 8 – TESTING

8.4 CRUD Testing

CRUD testing verifies creation, retrieval, updating, and permitted deletion of resources.

ID	Test	Expected Result	Actual Status
TC01	Valid request	Success response	Fill after execution
TC02	Invalid request	Validation error	Fill after execution
TC03	Unauthorized request	Access denied	Fill after execution
TC04	Database operation	Correct persistence	Fill after execution
TC05	AI request	Expected insight structure	Fill after execution

CHAPTER 8 – TESTING

8.5 Validation Testing

Validation testing verifies that incomplete, incorrectly formatted, or logically invalid requests are handled appropriately.

ID	Test	Expected Result	Actual Status
TC01	Valid request	Success response	Fill after execution
TC02	Invalid request	Validation error	Fill after execution
TC03	Unauthorized request	Access denied	Fill after execution
TC04	Database operation	Correct persistence	Fill after execution
TC05	AI request	Expected insight structure	Fill after execution

CHAPTER 8 – TESTING

8.6 Database Integration Testing

Database testing verifies that API operations produce the intended persistent changes and that retrieved information matches stored records.

ID	Test	Expected Result	Actual Status
TC01	Valid request	Success response	Fill after execution
TC02	Invalid request	Validation error	Fill after execution
TC03	Unauthorized request	Access denied	Fill after execution
TC04	Database operation	Correct persistence	Fill after execution
TC05	AI request	Expected insight structure	Fill after execution

CHAPTER 8 – TESTING

8.7 AI Module Testing

AI-module testing verifies that the module receives appropriate input, returns output in the expected structure, and communicates the informational nature of its insights.

ID	Test	Expected Result	Actual Status
TC01	Valid request	Success response	Fill after execution
TC02	Invalid request	Validation error	Fill after execution
TC03	Unauthorized request	Access denied	Fill after execution
TC04	Database operation	Correct persistence	Fill after execution
TC05	AI request	Expected insight structure	Fill after execution

CHAPTER 8 – TESTING

8.8 Test Case Summary

The following cases should be executed in the final project. The status must be updated using actual results rather than assumptions.

ID	Test	Expected Result	Actual Status
TC01	Valid request	Success response	Fill after execution
TC02	Invalid request	Validation error	Fill after execution
TC03	Unauthorized request	Access denied	Fill after execution
TC04	Database operation	Correct persistence	Fill after execution
TC05	AI request	Expected insight structure	Fill after execution

CHAPTER 9 – DEPLOYMENT

9.1 Deployment Approach

The backend can be executed locally during development and can later be deployed to a server or cloud environment. A production deployment should separate application configuration from source code.

42. Prepare environment.
43. Configure database.
44. Build application.
45. Start backend.
46. Test API.
47. Verify database.

CHAPTER 9 – DEPLOYMENT

9.2 Environment Configuration

Java, Maven, MySQL, and the application configuration are required. Database credentials should be maintained securely.

48. Prepare environment.
49. Configure database.
50. Build application.
51. Start backend.
52. Test API.
53. Verify database.

CHAPTER 9 – DEPLOYMENT

9.3 Database Configuration

MySQL must be available and the application must be configured with the correct database connection properties. The exact schema should match the final implementation.

54. Prepare environment.
55. Configure database.
56. Build application.
57. Start backend.
58. Test API.
59. Verify database.

CHAPTER 9 – DEPLOYMENT

9.4 Running the Application

A typical development flow is: install JDK, install Maven, start MySQL, configure the database, build the Maven project, start Spring Boot, and test APIs using Postman.

60. Prepare environment.
61. Configure database.
62. Build application.
63. Start backend.
64. Test API.
65. Verify database.

CHAPTER 10 – CHALLENGES AND LIMITATIONS

10.1 Development Challenges

The project involves several interacting technologies. Maintaining consistency between API requests, Java objects, database records, and AI processing is a major engineering challenge.

- Use modular architecture
- Validate input
- Use transactions
- Protect configuration
- Document actual implementation

CHAPTER 10 – CHALLENGES AND LIMITATIONS

10.2 Database Challenges

Relational modeling requires appropriate entities and relationships. Incorrect relationships can cause duplication or inconsistent information.

- Use modular architecture
- Validate input
- Use transactions
- Protect configuration
- Document actual implementation

CHAPTER 10 – CHALLENGES AND LIMITATIONS

10.3 Security and AI Challenges

Health information requires careful access control. AI-generated health information also needs clear limitations so that it is not mistaken for professional diagnosis.

- Use modular architecture
- Validate input
- Use transactions
- Protect configuration
- Document actual implementation

CHAPTER 10 – CHALLENGES AND LIMITATIONS

10.4 Current Limitations

The review material does not specify exact endpoint paths, exact SQL columns, exact AI algorithm, production hosting, or measured test results. These should be inserted from the final implementation rather than invented.

- Use modular architecture
- Validate input
- Use transactions
- Protect configuration
- Document actual implementation

CHAPTER 11 – FUTURE ENHANCEMENTS

11.1 Mobile and Web Applications

A complete web and mobile client can provide easier access to health records and make the REST backend directly usable by end users.

- Improved accessibility
- Automation
- Visualization
- Personalization
- Integration

CHAPTER 11 – FUTURE ENHANCEMENTS

11.2 Wearable Integration

Wearable devices could automatically collect supported vital measurements, reducing manual entry.

- Improved accessibility
- Automation
- Visualization
- Personalization
- Integration

CHAPTER 11 – FUTURE ENHANCEMENTS

11.3 Advanced AI Analytics

Advanced AI analytics could identify patterns in historical information and provide more personalized awareness-oriented insights.

- Improved accessibility
- Automation
- Visualization
- Personalization
- Integration

CHAPTER 11 – FUTURE ENHANCEMENTS

11.4 Reminders, Dashboards and Healthcare Integration

Future versions can include medication reminders, appointment reminders, charts, dashboards, and controlled integration with healthcare services.

- Improved accessibility
- Automation
- Visualization
- Personalization
- Integration

CHAPTER 12 – CONCLUSION

12.1 Project Summary

The AI Personal Health System provides a centralized backend for personal health-data management using Java, Spring Boot, REST APIs, MySQL, JPA/JDBC, Maven, Postman, and an AI-based insight concept.

CHAPTER 12 – CONCLUSION

12.2 Achievements

The project established the major modules, architecture, technology stack, database concept, REST communication model, validation and transaction approach, testing strategy, and future expansion plan.

CHAPTER 12 – CONCLUSION

12.3 Skills Learned

The project provides practical exposure to Java backend development, Spring Boot, REST API design, relational databases, persistence technologies, validation, testing, system architecture, teamwork, and responsible AI application.

CHAPTER 13 – REFERENCES

13.1 Technical References

66. Java documentation and Java Platform documentation.
67. Spring Boot official documentation.
68. MySQL official documentation.
69. Apache Maven documentation.
70. Jakarta Persistence / JPA documentation.
71. Java Database Connectivity documentation.
72. Postman documentation.

The above references correspond to the technologies named in the project review. The final submitted report should retain resources actually consulted by the team.

CHAPTER 13 – REFERENCES

13.2 Project and Academic References

73. AI Personal Health System – Project Review Presentation, Group 1, K L (Deemed to be) University, 2026.
74. Sample Project Documentation supplied for the prescribed chapter structure.
75. Project source code and database schema maintained by the project team.
76. Postman API collections created during project testing, where applicable.

APPENDICES

APPENDIX A – Team Details

The project team consists of G. Tej Sahith, A. Sreepad, and Kambhammettu Srithika.

Member	Register Number	Role
G. Tej Sahith	2520030606	Team Lead
A. Sreepad	2520030152	Backend & Database Developer
Kambhammettu Srithika	2520030361	AI & Health Module Developer

APPENDICES

APPENDIX B – Technology Summary

Java, Spring Boot, RESTful Web Services, MySQL, JPA/JDBC, Maven, VS Code/IntelliJ IDEA, and Postman form the documented technology stack.

Layer	Technology
Backend	Java + Spring Boot
API	RESTful Web Services
Database	MySQL
Data Access	JPA / JDBC
Build	Maven
Testing	Postman
AI	AI-based analysis

APPENDICES

APPENDIX C – API Documentation Template

Record each actual endpoint with HTTP method, path, purpose, authentication requirement, request body, response body, status codes, and database effect.

Field	Value
Endpoint	Actual path
Method	GET / POST / PUT / DELETE
Purpose	Description
Authentication	Required / Not Required
Request	Actual JSON
Response	Actual JSON
Status	Actual HTTP status

APPENDICES

APPENDIX D – Database Documentation Template

Record each actual table with field name, data type, key, nullability, default value, relationship, and description.

Field	Description
Table	Actual table name
Column	Actual column
Type	SQL data type
Key	Primary/Foreign/None
Nullable	Yes/No
Description	Meaning

APPENDICES

APPENDIX E – Postman Evidence

Insert screenshots of API requests and responses here. Include successful and validation/error cases.

77. Capture request URL.
78. Capture method and headers.
79. Capture request body.
80. Capture response.
81. Record HTTP status.
82. Insert screenshot with figure number.

APPENDICES

APPENDIX F – Database Evidence

Insert screenshots of MySQL schemas, tables, sample records, and relevant queries here.

Field	Description
Table	Actual table name
Column	Actual column
Type	SQL data type
Key	Primary/Foreign/None
Nullable	Yes/No
Description	Meaning

APPENDICES

APPENDIX G – Application Screenshots

Insert actual application screenshots here if a web or mobile client exists. Do not insert unrelated screenshots.

- 83. Login or registration screen if implemented.
- 84. Dashboard if implemented.
- 85. Health record screen if implemented.
- 86. API testing evidence.
- 87. AI insight output if implemented.

APPENDICES

APPENDIX H – Final Verification Checklist

Verify names, register numbers, guide name, exact API endpoints, database schema, screenshots, test results, implementation details, and references before submission.

- Confirm team names and register numbers.
- Confirm guide name.
- Confirm technology stack.
- Confirm database schema.
- Confirm actual API paths.
- Confirm actual testing results.
- Confirm screenshots and captions.
- Update table of contents.

FINAL SUBMISSION

Project Documentation Quality Checklist

- Use consistent Times New Roman formatting throughout the report.
- Apply Heading styles to chapter and section headings.
- Update the Table of Contents in Microsoft Word.
- Insert the final project screenshots in the appendices.
- Replace all “To be verified” testing statuses with actual results.
- Replace API placeholders with actual endpoint paths.
- Replace database documentation placeholders with the actual schema.
- Verify all names and register numbers.
- Verify the guide and department details.
- Ensure AI-health statements remain informational and do not claim professional diagnosis.
- Check page numbers, figure captions, table captions, and spelling.
- Export the final Word document to PDF after final verification.

This report is deliberately structured to exceed the requested 70-page length through section-level organization, detailed technical explanations, tables, architecture diagrams, testing documentation, and appendices. It should be synchronized with the actual source code before final submission.